

Predicting the Health Status of Ecological Sites Using Machine Learning Methods

Courtney G. Sweeney

Simmons University

CS 346

Professor Nanette Veilleux

May 8, 2026

I. Introduction

The “Projecting Responses of Ecological Diversity In Changing Terrestrial Systems” (PREDICTS) data set has over 1,000,000 entries and 72 attributes. Each row represents an observation related to a species at a site, recorded during a study. These rows are made unique by the time at which they were measured and the date. This data was created to track how human activity affects biodiversity. Multiple variables specifically highlight this and are crucial to the final models; these include *land use* and *use intensity*. The goal of these models is to classify sites as *Healthy*, *Transitional*, or *Degraded* using the attributes recorded in this study. By classifying sites, this model can help scientists identify when and why sites become degraded to intervene sooner to restore ecological diversity.

II. Data Cleaning

First, all character columns were converted to categorical variables. Then, every variable was evaluated for deletion.

Reference, *source for predominant land use*, and *habitat as described* were removed from the dataset because they are descriptive attributes containing only text that couldn't be converted into numerical attributes or useful levels. *Measurement*, *taxon name entered*, *parsed name*, *block*, *taxon number*, *SS*, *SSB*, *SSS*, *genus*, *species*, *phylum*, *site name*, *site number*, *source ID*, *study name*, *realm*, and *study number* were all removed for being redundant. The data captured in these variables is already captured with another variable, so they were all removed. *Transect details*, *source for predominant habitat*, *wilderness area*, *eco region distance metres*, *habitat patch area square metres*, *km to nearest edge of habitat*, *max linear extent*, and *predominant habitat* were all removed from the dataset because they were almost entirely composed of missing values.

Years since fragmentation seems like it should be a very important variable for this dataset. However, after research into the variable's missing values, it became clear that the missing data were not MCAR. If they were, it would have been possible to use some of the other attributes to impute values. Different strategies were considered for dealing with this variable, including SMOTE and mean imputation. However, because there was such a significant amount missing, and the data was not MCAR, this could have done more damage than good to the dataset and created signals that weren't there. If there had been enough information on this column, it likely would have been an incredible predictor of site health. The issue was apparent when looking at the missing values for this attribute grouped by land use. If the land was never used by humans, it has to have a missing value for years since fragmentation; if the land had always (relatively) been used by humans, there is also no possible value. *Primary vegetation* (never been used) had a 99% missing rate for this column, and *urban* (always been used) had around a 98% missing rate. There was only one land use type that had as low as a 20% missing rate for this attribute, and the total amount of missing data was around 500,000 rows. So, this column was deleted, even though it would have been an interesting column to look at.

Once these attributes were removed, the missing values were removed. *Cannot decide*, a level of predominant land use, was removed from the dataset, as it had an insignificant number

of rows and didn't provide any information about the site. Rows where *sampling effort* was "NA" were removed, which was not a significant number of rows. *Best guess binomial* and *family* had a high number of missing values, but because of the nature of these values, it would have been impractical to use the most common value of either of these, since they are dependent on what species was examined in that row. Every row where they were NA was removed, leading to a significant decrease in rows. However, because the initial dataset was so large, the dataset still had over 700,000 rows after this. Because this step also removed every other missing value, this step was kept after debating between doing this and attempting to match taxons with these values.

After this, the data had to be normalized. Effort-corrected measurement was an interesting case, as only 70,000 rows had non-zero entries for this column. To fix this, a column that is binary was added to signify whether the species was present. This column also had a significant amount of outliers, which were dependent on the sampling methods. However, these outliers were not thrown away, as they were features of the dataset, not mistakes. They were later dealt with after clustering, but it was important to keep them here. Then, to fix the skew, the attribute was log-transformed. This log-transformed attribute, along with *year*, *longitude*, and *latitude* were standardized using Z-score. The two true/false variables surrounding whether a diversity metric was suitable were transformed into 1's and 0's.

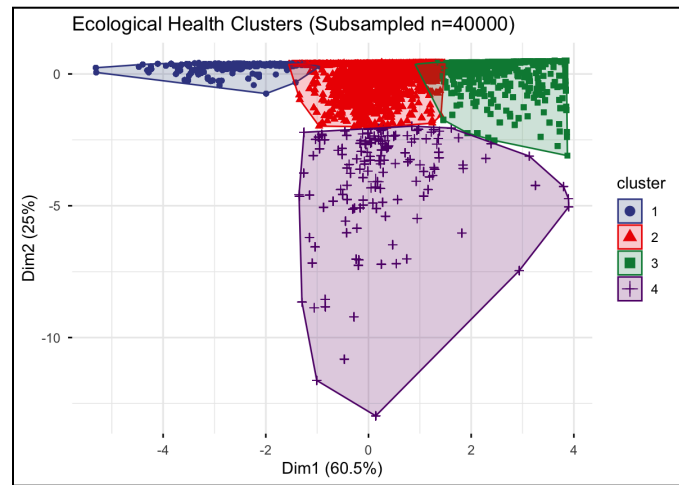
Predominant land use, use intensity, and *sampling method* were turned into indicator variables. There are fewer than 10 levels for each of these variables, so it was relatively easy to transform them into these columns. The original columns were removed after this. Some other columns were considered to be turned into indicator variables, including country, biome, and realm. However, these variables had too many levels, and a sparse matrix would have been created had this been done. Instead, these columns were turned into word embeddings for the neural network and support vector machine to use. By doing this, the meaning of the words was preserved without creating a sparse matrix.

III. Clustering

Initially, the classification levels were created using only the Shannon Index and the Simpson Index. The Shannon Index "measures the diversity of species in a community" (Bobbitt, 2022), and the Simpson Index is "a measure of diversity which takes into account the number of species present, as well as the relative abundance of each species." (*Simpson's Diversity Index*, 2025) Arbitrary thresholds were used to create different levels of Shannon and Simpson scoring, which ended up creating classes that might not have necessarily been grouped correctly. Shannon and Simpson indicate site health relatively well, but using these bounds created classes that were incredibly difficult to separate. To solve this problem, classification ended up being used instead.

Before classification, the data had to be split into different health levels. K-means clustering was used to determine these levels, based on the attributes of *site_simpson_index*, *site_shannon_index*, *site_total_abundance*, and *site_total_richness*. The four clusters formed

naturally based on these four attributes; the clustering method created two dimensions using these attributes, with one explaining 60.5% of the variance and the other 25%. After clustering was complete, each class had these stats averaged to understand why it was grouped. The first cluster had the second-best Shannon Index and Simpson Index averages, so it was labelled as transitional. The second cluster had the best Shannon and Simpson averages, so it was labelled as healthy. The third group was labeled degraded, having the worst averages. The fourth group, however, was close in Shannon and Simpson to the transitional group but had an incredibly high abundance average. This group caught all of the outliers from the effort-corrected measurement class, so it was thrown out. It only contained approximately 26,000 rows of data compared to the 700,000 rows contained in the other groups.



Health Status	Averages			
health_status	avg_shannon	avg_simpson	avg_abundance	avg_richness
1	0.5239471	0.2911697	2,187.002	3.080601
2	3.3346419	0.9414118	1,382.080	55.995928
3	2.0014464	0.8132311	2,738.989	11.207040
4	1.9719159	0.7759321	47,194.956	14.571577

Source: PREDICTS Database

IV. Random Forest and Attribute Selection

A decision tree was not able to handle the data in order to understand feature importance, so a Random Forest was used instead. The Random Forest not only revealed the most important attributes of the dataset, but also how balancing and class importance would affect accuracy in this dataset. After the first few runs of the Random Forest, it became clear that *month*, *year*, *longitude*, *latitude*, and *country* were correlated. A correlation matrix supported this, so only *country* was kept. *Sampling effort*, *rescaled sampling effort*, and *sampling effort unit* were also correlated with each other. These values were explained by the effort-corrected measurement attribute, so they were removed. The model also relied too heavily on *higher taxon*, which

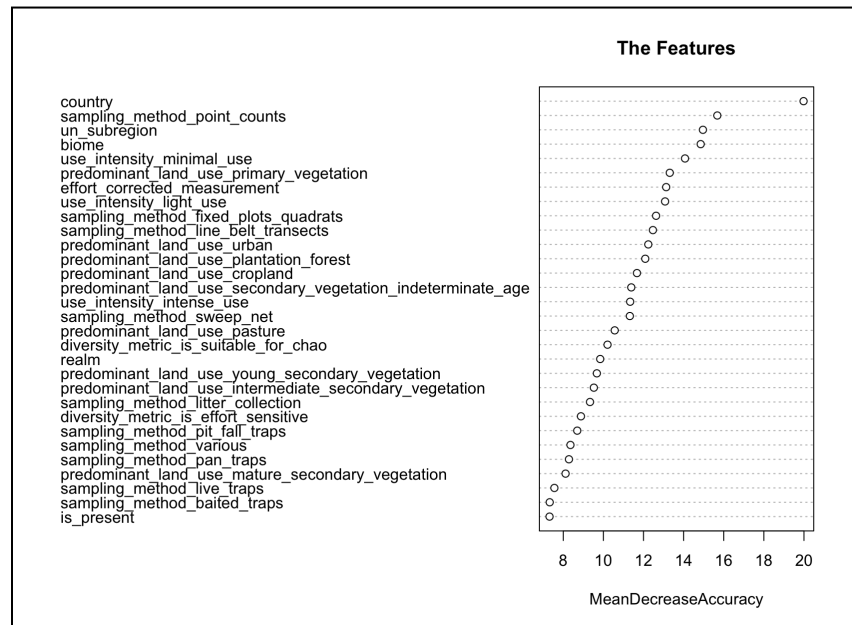
provides no information about the site itself, so it was removed. The most important attributes were revealed to be country, UN subregion, biome, predominant land use, use intensity, and sampling method.

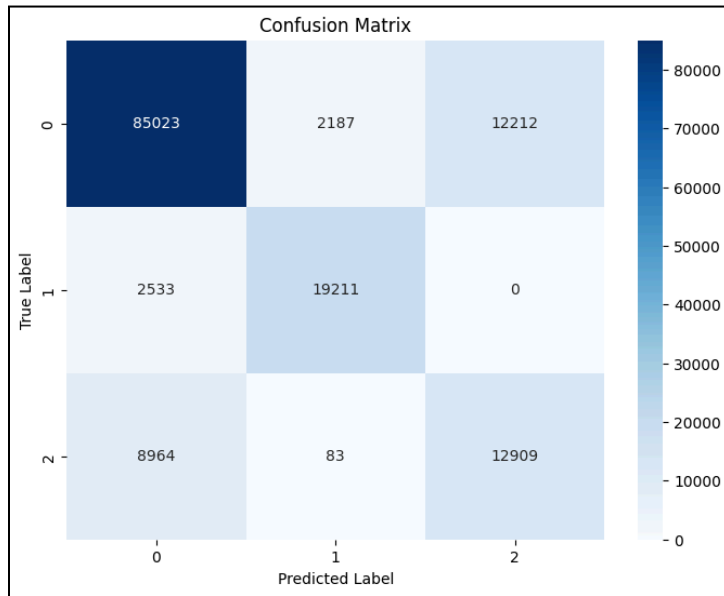
The data was balanced so that the Random Forest could take the first pass at seeing how well these attributes could be used for classification. With this balanced attempt, the accuracy was relatively low, but the minority class

(*degraded*) was classified very accurately. However, it was classifying too many instances as *degraded*, specifically from the *transitional* class. Because the *transitional* class is so large, it brought the accuracy down significantly. The next attempt to use a Random Forest was with unbalanced data in order to understand what was going on. This model had a significantly higher accuracy and an AUC of 0.9569, but a very low sensitivity for the *degraded* class. Because it is more important to catch degraded instances, the data needs to be balanced so that this doesn't occur.

V. Neural Network

The first model used with this data was a Neural Network. Because this data is complicated with attributes having different relationships with each other, a neural network was best suited to classify sites. It can capture complex relationships that humans cannot comprehend. The Neural Network was written in Google Colab in Python, which allows the user to power their models with stronger processing units. Before the neural network was created, biome, country, realm, and UN subregion were converted into word embeddings. The data was split into 80% training 20% testing. After being split, the training data was also balanced using undersampling of the majority class. The neural network used has three hidden layers: the first has 256 nodes, the second 128, and the third 64. Each layer uses the Exponential Linear Unit (ELU) activation function, which is a smooth version of ReLU. Dropout and batch normalization were also used. The output layer uses a softmax function to return class probabilities. The model was trained for around 50 epochs using a validation split, stopping when accuracy was no longer increasing by a substantial amount. The model ended up





classifying the test set with an accuracy of 0.85. Unfortunately, the model still ended up with a low recall for the degraded class. Around 13,000 instances of the degraded class were instead classified as transitional. When the model used class weights instead of balancing through undersampling, the recall for the degraded class was 0.82. This is not the ideal way to deal with class imbalance, but it worked well for this model.

The final iteration of this neural network combined the two techniques. This model had an overall accuracy of 0.82 on the

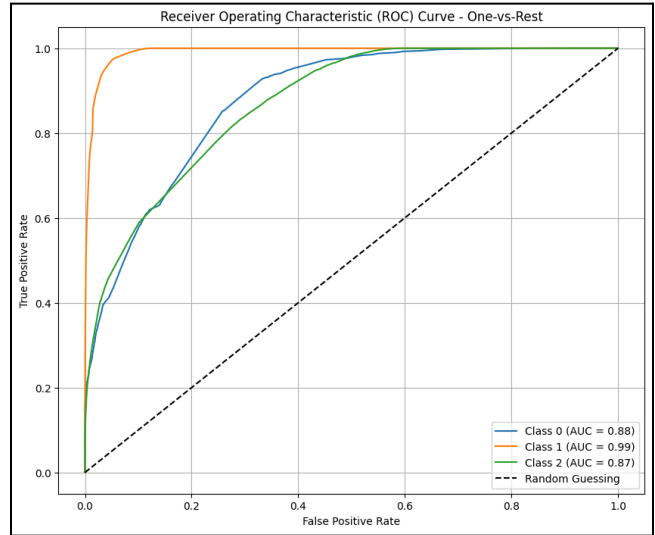
testing data and a degraded recall of 0.59. It still didn't manage to hit the degraded accuracy of the unbalanced model that only used class weights, but this model performed significantly better overall. Its overall accuracy of 0.82 is significantly higher than the unbalanced model's accuracy of 0.69, so the loss of recall for the degraded class might be worth it. This model primarily has issues mixing up the transitional class and the degraded class; thankfully, only 83 degraded instances out of 21,956 instances were classified as healthy. For future improvement, it might be beneficial to prioritize comparing the class probabilities created by the neural network, and looking closer at sites that are caught between classes, and comparing them manually, so that no degraded classes slip through the cracks.

VI. A Second Model

Originally, the second model to be compared to the neural network was a support vector machine. Unfortunately, R was unable to train an SVM on the data. Next, a decision tree was attempted. Even with fewer attributes, R was unable to train a decision tree. Finally, a Naive Bayes model was tested. Before the model was trained, the data was split into 70% training 30% testing, with the training data balanced after the split. The expectations for this model were low, and it performed extremely poorly. Not only was it worse than the neural network, but it also did significantly worse than guessing by chance. The no information rate was around 0.59, whereas the Naive Bayes model performed with an accuracy of 0.313. This model performed so poorly because it assumes feature independence, which is not the case for this data. It is important for models used on this set to understand the relationship between the attributes, as they operate together regarding the health of a site.

VII. Model Comparison and Evaluation

Overall, the neural network performed significantly better. It had a significantly better overall accuracy, with the final iteration of the neural network (the model that will be compared to Naive Bayes) having an accuracy of 0.82 compared to the 0.313 accuracy of the Naive Bayes model. Interestingly, Naive Bayes did the best (within itself) with the minority classes; degraded had a sensitivity of 0.56, and healthy had a sensitivity of 0.96. The reason that the Naive Bayes had a low accuracy was that it had a sensitivity of 0.11 for transitional. The model overpredicted healthy sites, which is a significant problem. While the goal was to have the highest recall/sensitivity for the degraded class, it is still important to catch transitional sites. These sites may also need help, so this value alone makes this model nearly useless. It would still be interesting if it had a higher recall for degraded sites than the neural network, but it doesn't. The main advantage that the Naive Bayes model has is the time that it takes to train it. The neural network took over twenty minutes to be trained, while the Naive Bayes model was trained in minutes. Unfortunately, the fact that the overall accuracy was 0.3 worse than the no information rate makes this advantage negligible. The Naive Bayes model was unable to capture the complex relationships between attributes that the Neural Network was able to, leading to it having a significantly lower accuracy.



Model Name	Accuracy	Degraded Sensitivity
Neural Network	0.82	0.59
Random Forest	0.688	0.8033
Naive Bayes	0.313	0.56

Source: PREDICTS Database

References

- Bobbitt, Z. (2022, April 20). *Shannon Diversity Index: Definition & Example*. Statology.
Retrieved May 9, 2026, from <https://www.statology.org/shannon-diversity-index/>
- Clustering in R | a guide to clustering analysis with popular methods*. (2022, October 10).
Domino Data Lab. Retrieved May 9, 2026, from <https://domino.ai/blog/clustering-in-r>
- Contu, S., De Palma, A., Bates, R., Borer, J., et al. (2022). *Release of data added to the PREDICTS database (November 2022)* [Data set]. Natural History Museum.
<https://doi.org/10.5519/jg7i52dg>
- Dubey, A., & Murtoff, J. (n.d.). *Species abundance | Definition, Conservation, & Facts*.
Britannica. Retrieved May 9, 2026, from
<https://www.britannica.com/science/species-abundance>
- Rafferty, J. P., & Johnston, R. J. (n.d.). *Species richness | Definition, Examples, & Facts*.
Britannica. Retrieved May 9, 2026, from
<https://www.britannica.com/science/species-richness>
- Simpson's Diversity Index*. (2025, October 25). Barcelona Field Studies Centre. Retrieved May 9, 2026, from <https://geographyfieldwork.com/Simpson'sDiversityIndex.htm>
- A single neuron neural network in Python*. (2025, September 30). GeeksforGeeks. Retrieved May 9, 2026, from
<https://www.geeksforgeeks.org/deep-learning/single-neuron-neural-network-python/>
- Skalski, P. (2018, October 12). *Let's code a Neural Network in plain NumPy*. Medium. Let's code a Neural Network in plain NumPy
- Visualize Clustering Results*. (n.d.). Factoextra. Visualize Clustering Results